:

Data Networks — Routing

2026-06-08

Table of contents

1 **1**

2 A. **2**
2.1 Optimality Principle Sink Tree . . . 2
2.2 Distance Vector = Bellman–Ford . . . 2
2.2.1 . . . 3
2.2.2 Count-to-Infinity . . . 3
2.3 Link State = Dijkstra . . . 4
2.4 . . . 4

3 B. **4**
3.1 . . . 4
3.1.1 . . . 5
3.2 . . . 6
3.3 Feasible / Descent Direction . . . 6
3.4 Frank–Wolfe (Flow Deviation) . . . 6
3.4.1 . . . 7
3.5 . . . 7
3.6 Gradient Projection . . . 7
3.6.1 . . . 7
3.6.2 FW vs Gradient Projection . . . 8
3.6.3 . . . 8
3.7 Two-Link : closed form . . . 10
3.7.1 . . . 10
3.7.2 . . . 12

4 C. **13**
4.1 Capacity Assignment . . . 13
4.2 Concentrator Location . . . 14

5 **14**

1

.

- **A** — : “ ” (Bellman–Ford, Dijkstra). .
- **B** — : “ .” .

B . A , B first-principles .

$$D_{ij}(F_{ij}) = \frac{F_{ij}}{C_{ij} - F_{ij}}$$ **M/M/1** $\rho/(1-\rho)$. $\sum_{(i,j)} D_{ij}$ Little

---

2 A.

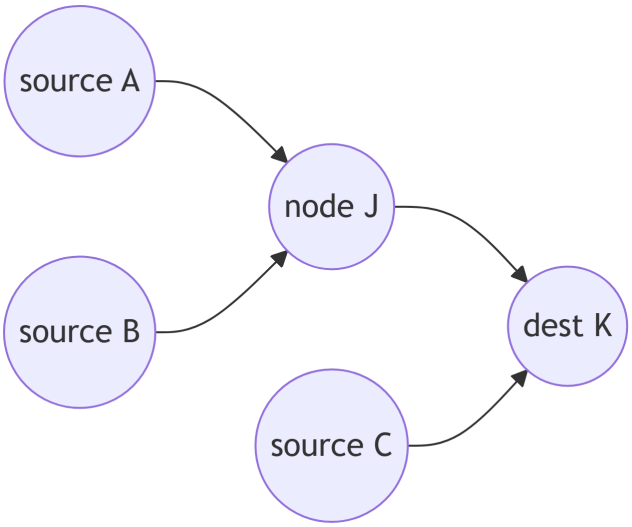
2.1 Optimality Principle Sink Tree

J $I \to K$, $J \to K$.

: K K **(sink tree)** . ,
( ). sink tree .

(Bellman) , Bellman–Ford .

```
graph LR
  S1((source A)) --> J((node J))
  S2((source B)) --> J
  J --> K((dest K))
  S3((source C)) --> K
```



2.2 Distance Vector = Bellman–Ford

(,) . 1 :

- $d_{ij} = \infty$: (i, j) arc
- D_i^h : i 1 h **arc**
- $D_1^h = 0$ $(\forall h)$, $D_i^0 = \infty$ $(\forall i \neq 1)$

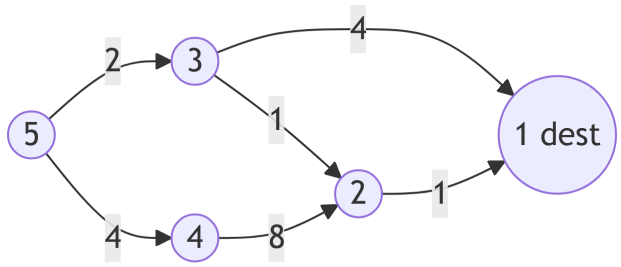
$$D_i^{h+1} = \min_j \left[d_{ij} + D_j^h \right], \qquad \forall i \neq 1$$

: " hop j (d_{ij}) + h hop (D_j^h)" . hop DP , $h = N - 1$.

2.2.1

1. 1 .

```
graph LR
  N2((2)) -->|1| N1((1 dest))
  N3((3)) -->|4| N1
  N3 -->|1| N2
  N4((4)) -->|8| N2
  N5((5)) -->|2| N3
  N5 -->|4| N4
```



.

```
import math
import pandas as pd

nodes = [1, 2, 3, 4, 5]
#   (1)   out-arc:  i -> (j,   d_ij)
arcs = {1: [], 2: [(1, 1)], 3: [(1, 4), (2, 1)], 4: [(2, 8)], 5: [(3, 2), (4, 4)]}

INF = math.inf
D = {i: (0 if i == 1 else INF) for i in nodes}
history = [dict(h=0, **{f"D{i}": D[i] for i in nodes})]

for h in range(1, 5):
    newD = {i: (0 if i == 1 else INF) for i in nodes}
    for i in nodes:
        if i == 1:
            continue
        cands = [d_ij + D[j] for (j, d_ij) in arcs[i]]
        newD[i] = min(cands) if cands else INF
    D = newD
    history.append(dict(h=h, **{f"D{i}": D[i] for i in nodes}))

pd.DataFrame(history).set_index("h")
```

$h = 1$, $h = 2$ 3 2 $4 \to 2$, $h = 3$ 5 3 $6 \to 4$ $h = 3$. “ ” **hop** .

2.2.2 Count-to-Infinity

( ) ( ) . $A-B$ C (A stale) “ B 2 ” , A “C 3” . C “ 4”... 1 ∞ .

h	D1	D2	D3	D4	D5
0	0	inf	inf	inf	inf
1	0	1.0	4.0	inf	inf
2	0	1.0	2.0	9.0	6.0
3	0	1.0	2.0	9.0	4.0
4	0	1.0	2.0	9.0	4.0

Figure 1

⚠ DV

1. (count-to-infinity): hop .
2. : → B .

2.3 Link State = Dijkstra

(1) , (2) , (3) , (4) **broadcast**, (5) **Dijkstra** .

- : $P = \{1\}$, $D_1 = 0$, $D_j = d_{j1}$ $(j \neq 1)$
- Step 1: P $D_i = \min_{j \notin P} D_j$ i P . .
- Step 2: P j $D_j := \min\left[D_j,\ d_{ji} + D_i\right]$. Step 1 .

greedy : arc , P i i (→). . DV hop DP .

Distance Vector (BF)	Link State (Dijkstra)
, , count-to-infinity , hop	

2.4

- **Hierarchical routing**: · · CPU · → .
- **Flooding**: . (a) hop counter 0 , (b) . · . DB .
- **Multicast**: (join/leave) , spanning tree . : DVMRP, Core Based Tree.

---

3 B.

3.1

: F_{ij}(flow), C_{ij}( ), d_{ij}( ), D_{ij}( ).

$$D_{ij}(F_{ij}) = \frac{F_{ij}}{C_{ij} - F_{ij}} + d_{ij}F_{ij}$$

M/M/1 $(\rho/(1-\rho), \rho = F_{ij}/C_{ij})$, Little “ ”$(d_{ij} \times F_{ij})$. , $\gamma = \sum_w r_w$.

: OD pair W, w P_w, flow x_p, flow r_w.

$$\sum_{p \in P_w} x_p = r_w \quad (\forall w), \qquad x_p \geq 0, \qquad F_{ij} = \sum_{p \ni (i,j)} x_p$$

:

$$\min_{\{x_p\}} \; D(x) = \sum_{(i,j)} D_{ij}\Big(\sum_{p \ni (i,j)} x_p \Big) \quad \text{s.t.} \quad \sum_{p \in P_w} x_p = r_w, \quad x_p \geq 0$$

: (1) flow , (2) , (3) **(splitting)**. A .

3.1.1

```
import numpy as np
import matplotlib.pyplot as plt

C = 1.0
F = np.linspace(0, 0.98 * C, 400)
D = F / (C - F)
Dp = C / (C - F) ** 2

fig, ax = plt.subplots(1, 2, figsize=(10, 3.8))
ax[0].plot(F / C, D, lw=2)
ax[0].set(xlabel=r"utilization $\rho=F/C$", ylabel=r"$D(F)=F/(C-F)$",
          title="Link cost (mean # in system)")
ax[0].grid(alpha=0.3)
ax[1].plot(F / C, Dp, lw=2, color="C3")
ax[1].set(xlabel=r"utilization $\rho=F/C$", ylabel=r"$D'(F)=C/(C-F)^2$",
          title="Marginal delay (per-link DFDL term)")
ax[1].grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

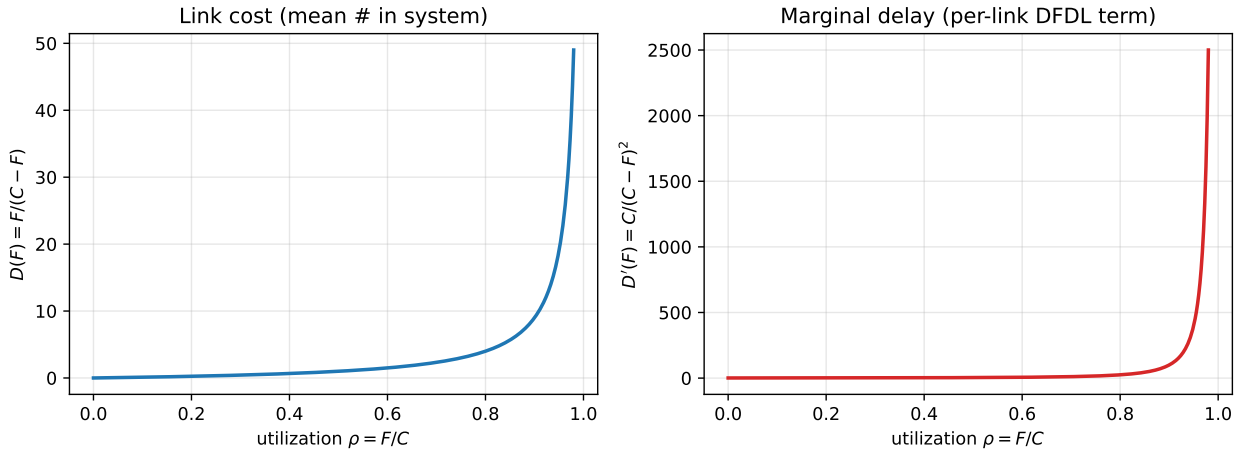


Figure 2: M/M/1 D(F) D'(F). 1 .

3.2

p **First Derivative Length (DFDL, 1):**

$$\frac{\partial D(x)}{\partial x_p} = \sum_{(i,j)\in p} D'_{ij}(F_{ij})$$

“ p flow ”, **(marginal delay)** . .

: x^* OD pair p flow δ p' . 1

$$\delta \frac{\partial D(x^*)}{\partial x_{p'}} - \delta \frac{\partial D(x^*)}{\partial x_p} \geq 0.$$

!

OD pair w flow $p \in P_w$

$$x_p^* > 0 \implies \frac{\partial D(x^*)}{\partial x_p} \leq \frac{\partial D(x^*)}{\partial x_{p'}}, \quad \forall p' \in P_w.$$

flow **(DFDL)** **, flow 0** **DFDL** **.**

: , . ( ) — , Wardrop , KKT . (system optimal) .

3.3 Feasible / Descent Direction

x Δx:

- **Feasible**: $x + \alpha\Delta x$ $\alpha > 0$ $\rightarrow$ OD pair $\sum_{p\in P_w} \Delta x_p = 0$, $x_p = 0$ $\Delta x_p \geq 0$.
- **Descent**:

$$\sum_{w\in W}\sum_{p\in P_w} \frac{\partial D(x)}{\partial x_p}\Delta x_p < 0, \qquad \frac{\partial D(x)}{\partial x_p} = \sum_{(i,j) \text{ on } p} D'_{ij}(F_{ij}).$$

step size $\alpha > 0$ $D(x + \alpha\Delta x) < D(x)$. 1 “feasible + descent + step size” .

3.4 Frank–Wolfe (Flow Deviation)

$x = \{x_p\}$:

1. OD pair **MFDL(Min First Derivative Length)** (D'_{ij} Dijkstra).
2. MFDL $\bar{x} = \{\bar{x}_p\}$.
3. : $\alpha^* \in [0,1]$: $D[x + \alpha^*(\bar{x} - x)] = \min_{\alpha\in[0,1]} D[x + \alpha(\bar{x} - x)]$.
4. : $x_p := x_p + \alpha^*(\bar{x}_p - x_p)$ $(\forall p)$.

: D 1 , (simplex) (“ MFDL ”) . α^* .

i

MFDL α^* (equal-ratio shift). .

3.4.1

$$D(x) = \tfrac{1}{2}(x_1^2 + x_2^2 + 0.1x_3^2) + 0.55x_3, \qquad x_1 + x_2 + x_3 = 1,\ x_i \geq 0$$

$$\frac{\partial D}{\partial x_1} = x_1, \quad \frac{\partial D}{\partial x_2} = x_2, \quad \frac{\partial D}{\partial x_3} = 0.1x_3 + 0.55$$

3 0.55 , . $x_3^* = 0,\ x_1^* = x_2^* = \frac{1}{2}$, .

3.5

— **steepest descent** ($\nabla^2 f$):

$$x^{k+1} = x^k - \alpha^k \nabla f(x^k), \qquad \frac{f(x^{k+1}) - f^*}{f(x^k) - f^*} \leq \left(\frac{M-m}{M+m}\right)^2$$

M, m Hessian · . M/m ( ) . B^k :

$$x^{k+1} = x^k - \alpha^k B^k \nabla f(x^k)$$

$B^k = [\nabla^2 f(x^k)]^{-1}$ **Newton's method**( ). Hessian :

$$x_i^{k+1} = x_i^k - \alpha^k \Big[\frac{\partial^2 f(x^k)}{\partial x_i^2}\Big]^{-1} \frac{\partial f(x^k)}{\partial x_i}$$

Positive orthant ($x \geq 0$): 0 projection $[\cdot]^+ = \max\{0, \cdot\}$:

$$x_i^{k+1} = \max\left\{0,\ x_i^k - \alpha^k \Big[\frac{\partial^2 f(x^k)}{\partial x_i^2}\Big]^{-1} \frac{\partial f(x^k)}{\partial x_i}\right\}$$

3.6 Gradient Projection

: $\sum_p x_p = r_w$ **MFDL** p_w **flow** .

$$x_{p_w} = r_w - \sum_{\substack{p \in P_w \\ p \neq p_w}} x_p$$

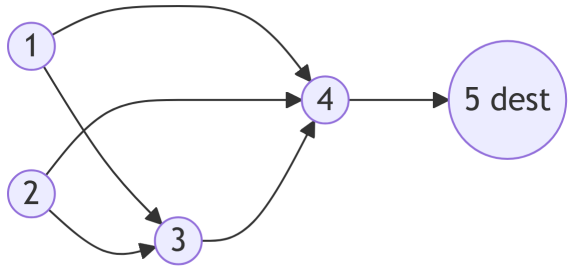
flow + positive orthant .

> **!** flow
>
> $$x_p^{k+1} = \max\left\{0,\ x_p^k - \alpha^k H_p^{-1}(d_p - d_{p_w})\right\}$$
>
> $$d_p = \sum_{(i,j)\in p} D'_{ij}(F_{ij}), \quad d_{p_w} = \sum_{(i,j)\in p_w} D'_{ij}(F_{ij}), \quad H_p = \sum_{(i,j)\in L_p} D''_{ij}(F_{ij})$$
>
> $L_p =$ p p_w ( ). flow .

: $(d_p - d_{p_w})$ " " , flow H_p^{-1} (2) step . FW (unequal shift) → .

3.6.1

```
graph LR
  N1((1)) --> N3((3))
  N1 --> N4((4))
  N2((2)) --> N3
  N2 --> N4
  N3 --> N4
  N4 --> N5((5 dest))
```



OD (1,5): $r_1 = 4$, OD (2,5): $r_2 = 8$. $D_{ij}(F_{ij}) = \frac{1}{2}F_{ij}^2 \rightarrow D'_{ij} = F_{ij},\ D''_{ij} = 1$.

$p_1(1) = \{1,4,5\},\ p_2(1) = \{1,3,4,5\}$ / $p_1(2) = \{2,4,5\},\ p_2(2) = \{2,3,4,5\}$. flow $F_{13} = 4,\ F_{14} = 0,\ F_{23} = 8,\ F_{24} = 0,\ F_{34} = 12,\ F_{45} = 12$.

$$d_{p_1(1)} = F_{14} + F_{45} = 0 + 12 = 12, \qquad d_{p_2(1)} = F_{13} + F_{34} + F_{45} = 4 + 12 + 12 = 28$$

MFDL $p_1(1)(=12)$. flow $p_2(1)$ 4 . $L_p = \{(1,3),(3,4)\} \cup \{(1,4)\} = \{(1,3),(3,4),(1,4)\} \rightarrow H_p = 1 + 1 + 1 = 3$.

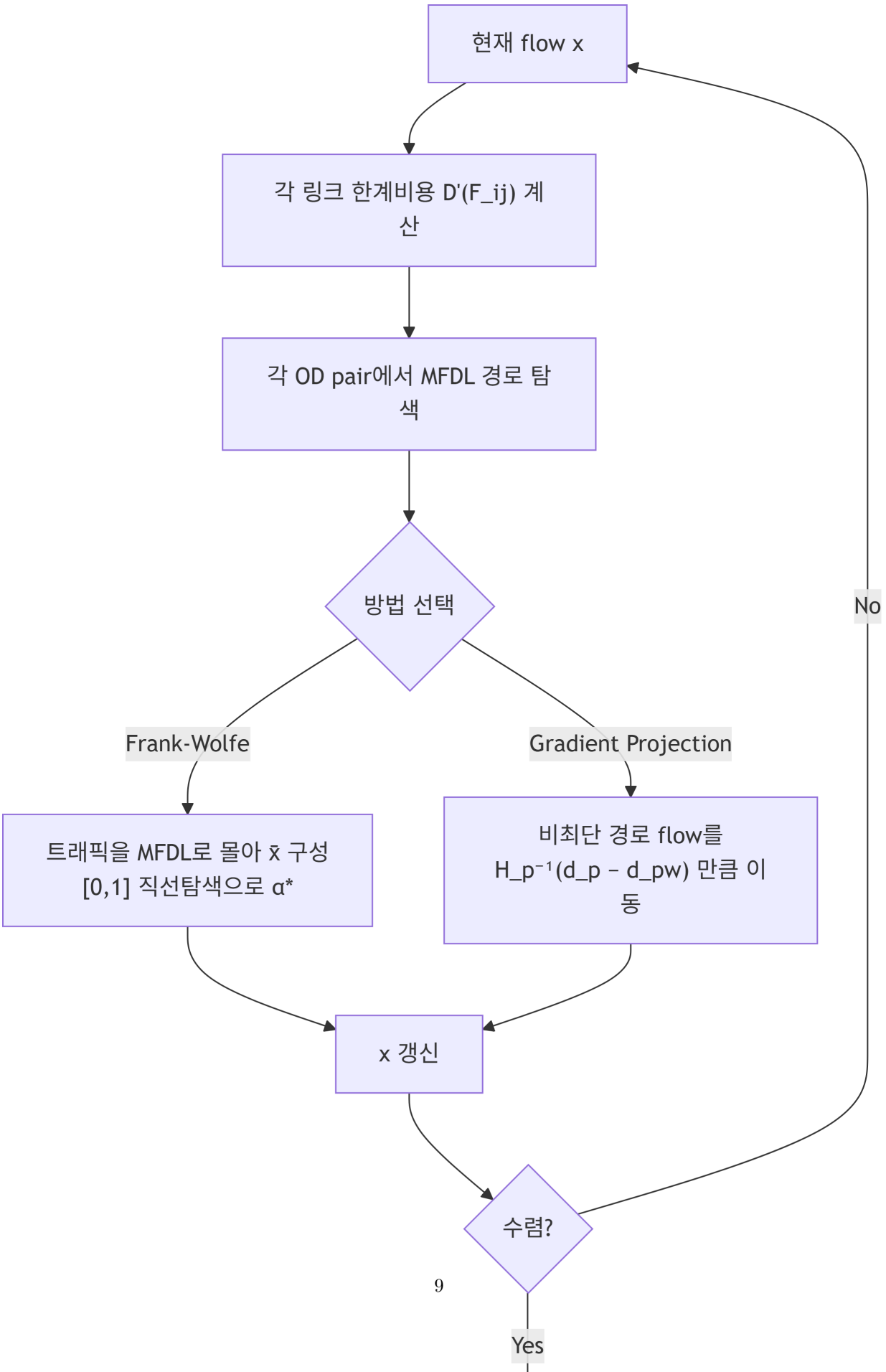
$$x_{p_2(1)} := \max\left\{0,\ 4 - \frac{\alpha^k}{3}(28 - 12)\right\} = \max\left\{0,\ 4 - \frac{16\alpha^k}{3}\right\}, \qquad x_{p_1(1)} := r_1 - x_{p_2(1)}$$

3.6.2 FW vs Gradient Projection

	Frank–Wolfe	Gradient Projection
	1 (DFDL)	$1 \cdot 2$ (H_p)
flow	MFDL	
	( )	
	(LP+)	(Hessian)

3.6.3

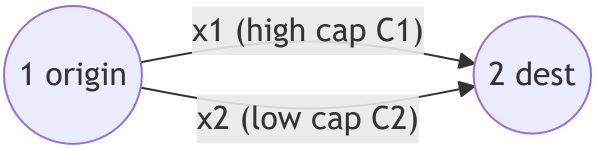
```
flowchart TD
  A["  flow x"] --> B["       D'(F_ij)  "]
  B --> C["  OD pair  MFDL    "]
  C --> Dn{"   "}
  Dn -->|"Frank-Wolfe"| E["   MFDL   x̄  <br/>[0,1]      *"]
  Dn -->|"Gradient Projection"| F["    flow <br/>H_p ¹(d_p - d_pw)    "]
  E --> G["x  "]
  F --> G
  G --> H{" ?"}
  H -->|"No"| A
  H -->|"Yes"| I["      "]
```


현재 flow x
각 링크 한계비용 D'(F_ij) 계산
각 OD pair에서 MFDL 경로 탐색
방법 선택
Frank-Wolfe
Gradient Projection
트래픽을 MFDL로 몰아 x̄ 구성
[0,1] 직선탐색으로 α*
비최단 경로 flow를 H_p⁻¹(d_p − d_pw) 만큼 이동
x 갱신
수렴?
No
Yes

3.7 Two-Link : closed form

OD , (C_1, C_2), r.

```
graph LR
  O((1 origin)) -->|"x1  (high cap C1)"| Dst((2 dest))
  O -->|"x2  (low cap C2)"| Dst
```



$$D(x) = D_1(x_1) + D_2(x_2), \quad D_i(x_i) = \frac{x_i}{C_i - x_i}, \quad x_1^* + x_2^* = r,\ x_i^* \geq 0$$

( ):

$$D_i'(x_i) = \frac{d}{dx_i}\frac{x_i}{C_i - x_i} = \frac{(C_i - x_i) + x_i}{(C_i - x_i)^2} = \frac{C_i}{(C_i - x_i)^2}$$

Case 1 — $(x_1^* = r,\ x_2^* = 0)$: (1) DFDL (2) DFDL:

$$\frac{C_1}{(C_1 - r)^2} \leq \frac{C_2}{C_2^2} = \frac{1}{C_2} \ \Rightarrow\ C_1 C_2 \leq (C_1 - r)^2 \ \Rightarrow\ \boxed{r \leq C_1 - \sqrt{C_1 C_2}}$$

> r . $1/C_2$, r $r^* = C_1 - \sqrt{C_1 C_2}$.

Case 2 — $(x_1^*, x_2^* > 0)$: DFDL .

$$\frac{C_1}{(C_1 - x_1^*)^2} = \frac{C_2}{(C_2 - x_2^*)^2} \ \Rightarrow\ \frac{\sqrt{C_1}}{C_1 - x_1^*} = \frac{\sqrt{C_2}}{C_2 - x_2^*}$$

$x_2^* = r - x_1^*$, x_1^* :

$$x_1^*(\sqrt{C_1} + \sqrt{C_2}) = \sqrt{C_1}[r - (C_2 - \sqrt{C_1 C_2})]$$

$$\boxed{x_1^* = \frac{\sqrt{C_1}[r - (C_2 - \sqrt{C_1 C_2})]}{\sqrt{C_1} + \sqrt{C_2}}, \qquad x_2^* = \frac{\sqrt{C_2}[r - (C_1 - \sqrt{C_1 C_2})]}{\sqrt{C_1} + \sqrt{C_2}}} \quad (r > C_1 - \sqrt{C_1 C_2})$$

($r = r^*$ $x_2^* = 0,\ x_1^* = r^*$ Case 1 .)

3.7.1

```
import numpy as np
import matplotlib.pyplot as plt

C1, C2 = 10.0, 4.0
r_star = C1 - np.sqrt(C1 * C2)

def split(r):
```

```
    if r <= r_star:
        return r, 0.0
    s1, s2 = np.sqrt(C1), np.sqrt(C2)
    x1 = s1 * (r - (C2 - np.sqrt(C1 * C2))) / (s1 + s2)
    x2 = s2 * (r - (C1 - np.sqrt(C1 * C2))) / (s1 + s2)
    return x1, x2

rs = np.linspace(0, 0.95 * (C1 + C2), 400)
X1 = np.array([split(r)[0] for r in rs])
X2 = np.array([split(r)[1] for r in rs])

plt.figure(figsize=(7, 4.2))
plt.plot(rs, X1, lw=2, label=r"$x_1^*$  (high cap $C_1=10$)")
plt.plot(rs, X2, lw=2, label=r"$x_2^*$  (low cap $C_2=4$)")
plt.axvline(r_star, ls="--", color="gray")
plt.text(r_star + 0.15, 8.4, rf"$r^*={r_star:.2f}$", color="gray")
plt.xlabel("total demand  r")
plt.ylabel("optimal path flow")
plt.title("Two-link optimal routing")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

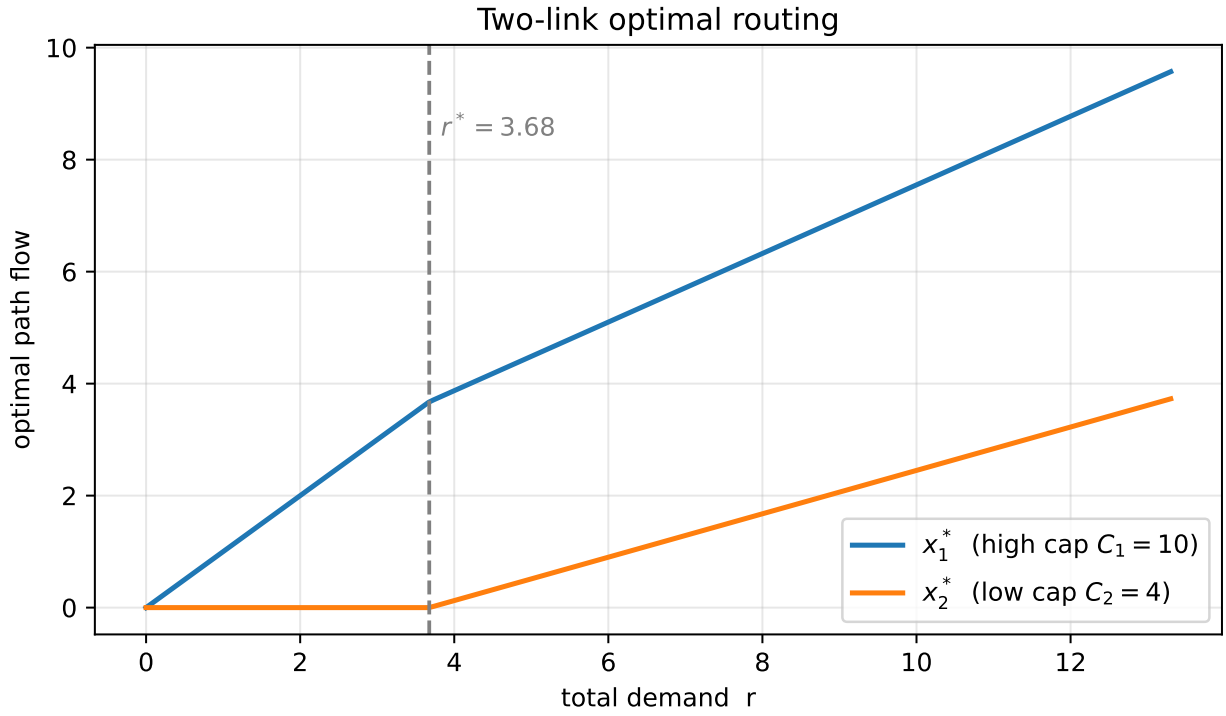


Figure 3: . r r* .

3.7.2

```
import numpy as np
import matplotlib.pyplot as plt

C1, C2, r = 10.0, 4.0, 6.0
x1 = np.linspace(r - C2 + 1e-3, min(r, C1) - 1e-3, 400)
x2 = r - x1
Dtot = x1 / (C1 - x1) + x2 / (C2 - x2)

s1, s2 = np.sqrt(C1), np.sqrt(C2)
x1_opt = s1 * (r - (C2 - np.sqrt(C1 * C2))) / (s1 + s2)
#
g1 = C1 / (C1 - x1_opt) ** 2
g2 = C2 / (C2 - (r - x1_opt)) ** 2
print(f"x1* = {x1_opt:.4f},  x2* = {r - x1_opt:.4f}")
print(f"DFDL link1 = {g1:.5f},  DFDL link2 = {g2:.5f}  (   )")

plt.figure(figsize=(7, 4.2))
plt.plot(x1, Dtot, lw=2)
plt.axvline(x1_opt, ls="--", color="C3", label=rf"$x_1^*={x1_opt:.3f}$")
plt.xlabel(r"$x_1$  (with $x_2 = r - x_1$)")
plt.ylabel(r"total cost  $D(x_1)+D(x_2)$")
plt.title(f"Total delay vs split at r={r}")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

```
x1* = 5.0994,  x2* = 0.9006
DFDL link1 = 0.41639,  DFDL link2 = 0.41639  (   )
```

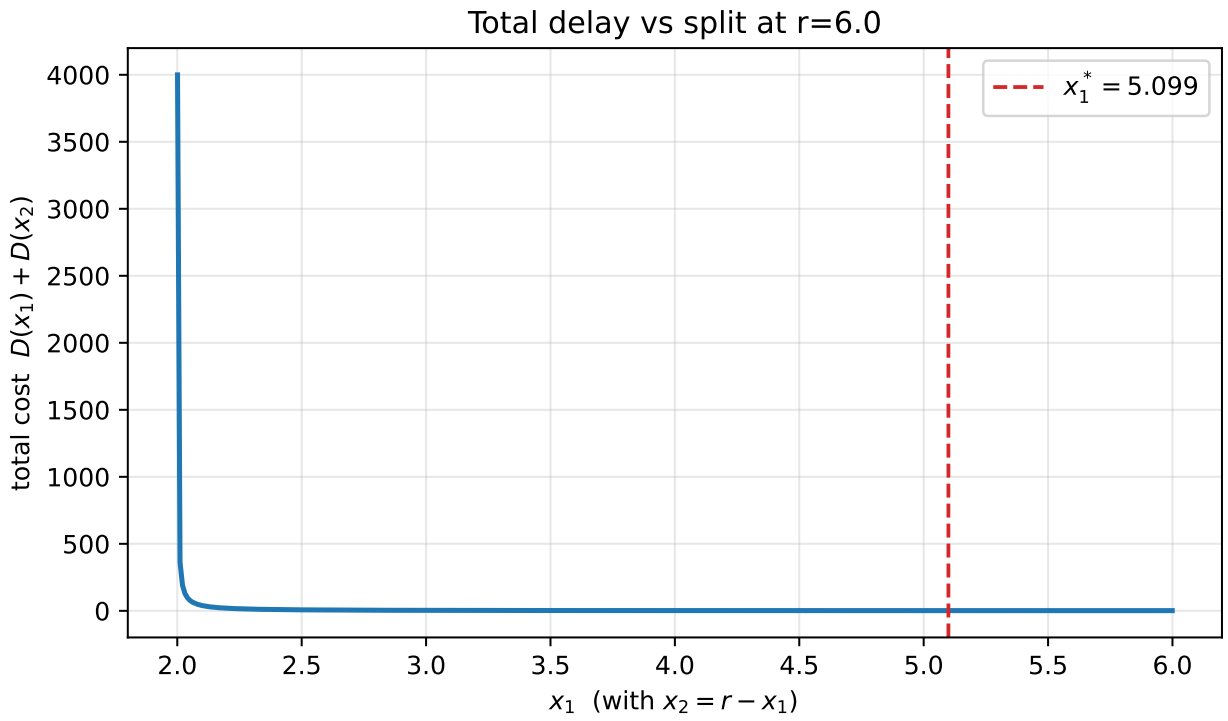


Figure 4: r x1 . .

($D_p(x_p) = \dfrac{x_p}{C_p - x_p}$) . (1) $D_p'(x_p) = \dfrac{C_p}{(C_p - x_p)^2}$ → (2) “ ” case → (3) DFDL $\sum_p x_p = r$. $C_1 > C_2 > C_3$ r .

---

4 C.

4.1 Capacity Assignment

OD flow F_{ij} , p_{ij} T :

$$\min \sum_{(i,j)} p_{ij} C_{ij} \quad \text{s.t.} \quad \frac{1}{\gamma} \sum_{(i,j)} \frac{F_{ij}}{C_{ij} - F_{ij}} \le T$$

Lagrangian $L = \sum_{(i,j)} \left(p_{ij} C_{ij} + \dfrac{\beta F_{ij}}{\gamma (C_{ij} - F_{ij})} \right)$ $\partial L / \partial C_{ij} = 0$:

$$C_{ij} = F_{ij} + \sqrt{\frac{\beta F_{ij}}{\gamma p_{ij}}}, \qquad \beta = \left(\frac{1}{T\gamma} \sum_{(i,j)} \sqrt{p_{ij} F_{ij}} \right)^2$$

: = “flow + ” , $\sqrt{F_{ij}/p_{ij}}$ — , (**square-root capacity assignment**).

4.2 Concentrator Location

i j $x_{ij} \in \{0,1\}$:

$$\min \sum_i \sum_j a_{ij} x_{ij} \quad \text{s.t.} \quad \sum_j x_{ij} = 1 \ (\forall i), \quad \sum_i x_{ij} \le K_j$$

simplex , $y_j \in \{0,1\}$ b_j (facility location) .

5

!

1. : “ (DFDL) , .” · .
2. **Two/three-link M/M/1 closed form**: $D'_p = \dfrac{C_p}{(C_p - x_p)^2}$ → case → .
3. **FW vs Gradient Projection**: vs , 1 vs 2 , MFDL .
4. **DV vs LS**: · · count-to-infinity .
5. **M/M/1 Little** .

Optimality principle	→ sink tree
Bellman–Ford	hop DP, $D_i^{h+1} = \min_j [d_{ij} + D_j^h]$, count-to-infinity
Dijkstra	greedy ( )
	,
DFDL	
Frank–Wolfe	→ MFDL → (equal-ratio)
Gradient projection	2 unequal shift,
Two-link	$r \le C_1 - \sqrt{C_1 C_2}$
Capacity assignment	square-root